



Zarządzanie procesem produkcji oprogramowania

Lab II

Mgr inż. Marek Andryszczyk

Wprowadzenie

Metodyka Agile (z angielskiego „zwinna”) powstała jako odpowiedź na trudności, jakie zespoły deweloperskie napotykały, korzystając z tradycyjnych metod wytwarzania oprogramowania, takich jak model kaskadowy (ang. *waterfall*). Tradycyjne podejścia, oparte na długoterminowym planowaniu i sztywnym podziale etapów, często prowadziły do opóźnień, przekraczania budżetów oraz finalnego produktu, który nie spełniał oczekiwań klientów.

Wprowadzenie

Przełomowy moment dla Agile miał miejsce w lutym 2001 roku, kiedy to 17 ekspertów z branży IT spotkało się w Snowbird, Utah, aby przedyskutować alternatywy dla ciężkich, formalnych metod pracy nad projektami informatycznymi. Efektem tego spotkania było stworzenie **Manifestu Agile** (*Agile Manifesto*), który stał się fundamentem podejścia Agile.

Cechy Agile



Manifest Agile

Manifest Agile składa się z czterech głównych wartości i dwunastu zasad:

Cztery wartości Agile:

- 1. Ludzie i interakcje** ponad procesy i narzędzia.
- 2. Działające oprogramowanie** ponad szczegółową dokumentację.
- 3. Współpraca z klientem** ponad negocjacje kontraktowe.
- 4. Reagowanie na zmiany** ponad realizację założonego planu.

Zasady Agile

1. Najwyższym priorytetem jest zadowolenie klienta poprzez wczesne i ciągłe dostarczanie wartościowego oprogramowania.
2. Zmiany wymagań są naturalne, nawet w późnych etapach projektowania.
3. Dostarczanie działającego oprogramowania regularnie, co kilka tygodni lub miesięcy.
4. Bliska współpraca biznesu i deweloperów przez cały czas trwania projektu.
5. Projekty powinny być prowadzone przez zmotywowane jednostki, którym daje się zaufanie.
6. Najlepszym sposobem przekazywania informacji w zespole jest bezpośrednia rozmowa.
7. Działające oprogramowanie to podstawowy miernik postępu.
8. Zwinne procesy promują zrównoważony rozwój.
9. Ciągła dbałość o doskonałość techniczną i dobre projektowanie.
10. Prostota, czyli sztuka maksymalizacji ilości pracy, która nie musi być wykonana, jest kluczowa.
11. Najlepsze architektury, wymagania i projekty pochodzą od samoorganizujących się zespołów.
12. Zespół regularnie analizuje, jak jeszcze bardziej usprawnić swoje działanie, i dostosowuje się do tych wniosków.

Kiedy stosować Agile

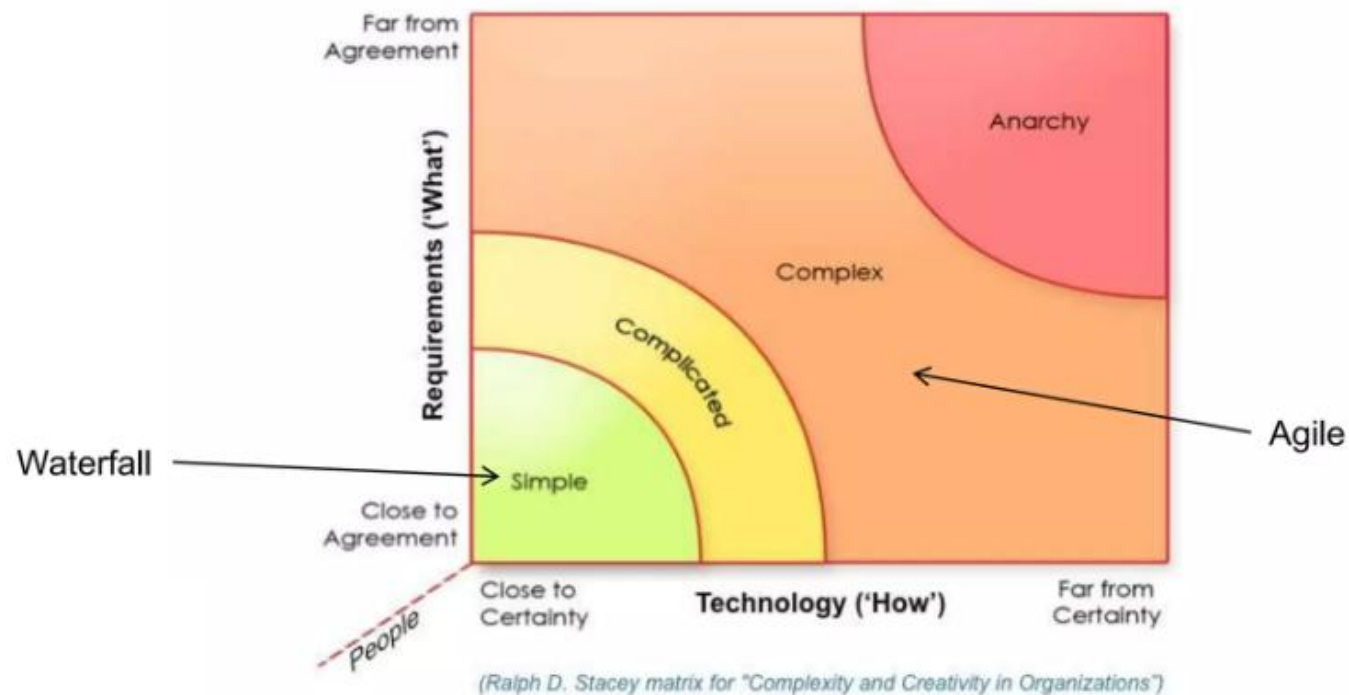
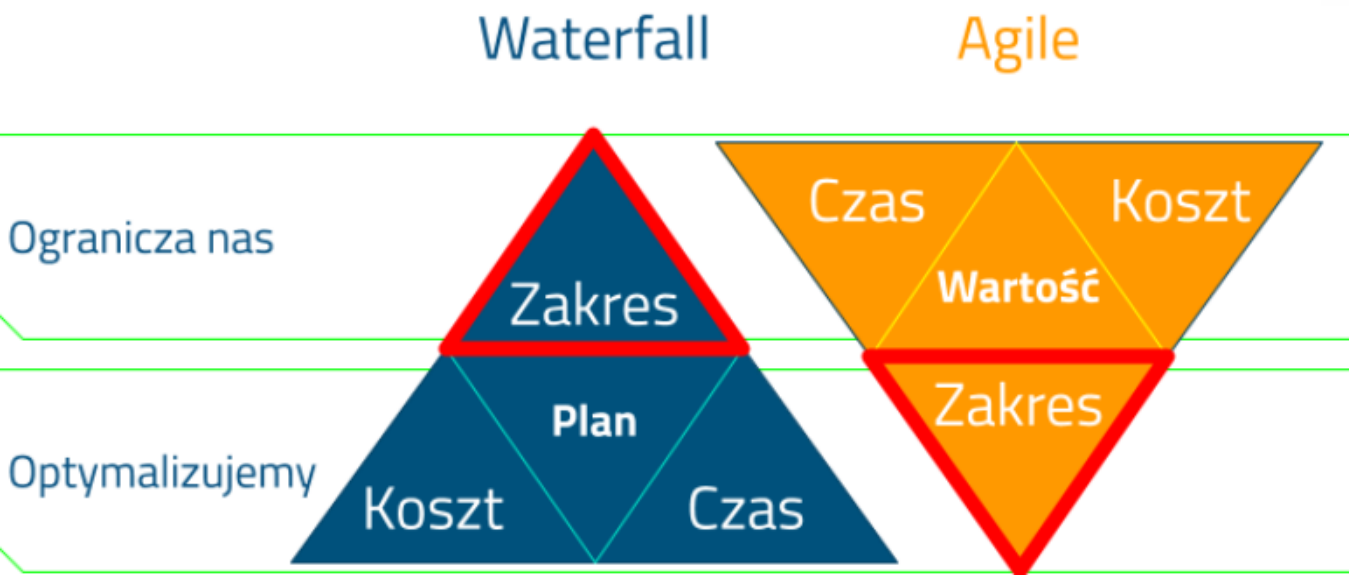
1. Klasyka Agile

1. „Ruchomy cel”, np. prace badawcze
2. Nieznany/niedookreślony efekt końcowy

2. Kiedy faktycznie stosować Agile:

1. Cel jest znany i może być bardzo dobrze określony
2. ...ale wiemy, że na pewno ulegnie zmianie ;)
3. Kiedy rezultat projektu jest bardzo trudny/kosztowny do modyfikacji (vs łatwy/możliwy do zmiany po koszcie zbliżonym do pierwotnego)

Poziom skomplikowania projektów

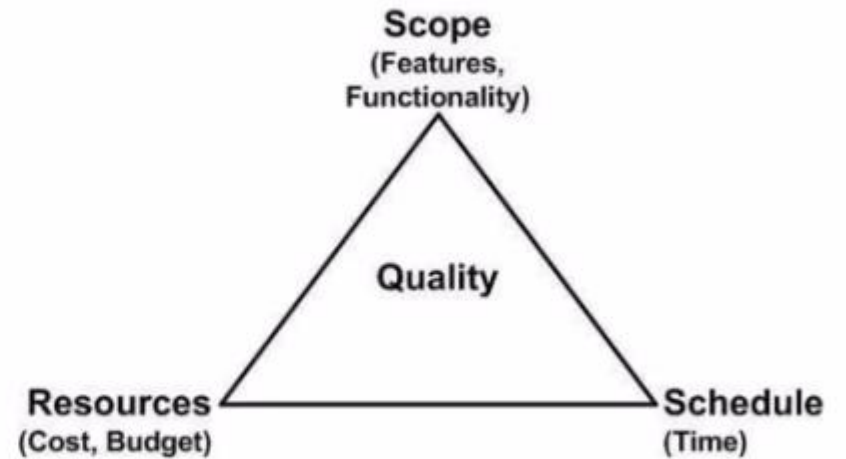


Metodologia projektowania oprogramowania

- Metodologia Agile to podejście do wytwarzania oprogramowania, które koncentruje się na elastyczności, współpracy z klientem i szybkim dostarczaniu wartościowych funkcji. .
 - **Iteracyjny rozwój** – Agile opiera się na krótkich cyklach rozwojowych zwanych sprintami (zwykle 1-4 tygodni). Każdy sprint kończy się dostarczeniem działającego fragmentu oprogramowania, co pozwala na częste dostarczanie wartości dla użytkownika.
 - **Współpraca z klientem** – W Agile klient jest aktywnie zaangażowany w proces tworzenia oprogramowania. Regularne spotkania i prezentacje wyników pracy (demonstracje) pozwalają na bieżąco dostosowywać produkt do potrzeb klienta.
 - **Zmieniające się wymagania** – Agile zakłada, że wymagania mogą się zmieniać w trakcie projektu. Zamiast sztywnego trzymania się początkowych ustaleń, zespół reaguje na zmiany i dostosowuje oprogramowanie, aby jak najlepiej odpowiadało potrzebom użytkowników.
 - **Samozarządzające się zespoły** – Zespoły Agile mają dużą autonomię i są odpowiedzialne za organizację własnej pracy. Współpraca między programistami, testerami, analitykami i innymi członkami zespołu jest kluczowa.
 - **Częste dostarczanie** – Celem Agile jest szybkie dostarczanie działających części oprogramowania. Zamiast czekać na zakończenie całego projektu, zespoły regularnie oddają klientowi gotowe funkcje, które można od razu wdrożyć.
 - **Stała poprawa** – Po każdym sprincie zespoły Agile przeprowadzają retrospektywy, aby zidentyfikować, co można poprawić w kolejnych iteracjach. Jest to kluczowy element ciągłego doskonalenia.

Ograniczenia

4 podstawowe ograniczenia każdego projektu:



Copyright 2003-2006 Scott W. Ambler

Zmieniając jeden z parametrów należy zmienić pozostałe, w przeciwnym razie nie dotrzemy parametru środkowego

Macierz kompromisów projektowych

Parametr	Ustalony (dokładnie 1)	Optymalizowany (dokładnie 1)	Negocjowany (pozostałe 2)
Czas			X
Budżet		X	
Zakres	X		
Jakość			X
	Kluczowy parametr, nie podlegający żadnym negocjacjom w trakcie trwania projektu	Np. realizując projekt jak najszybciej, najtaniej, jak najwięcej prac lub dbając o jakość.	Te dwa parametry mogą się zmieniać o określonych granicach

Metody Agile

1. Scrum: Struktura pracy, która dzieli projekt na tzw. sprinty – krótkie cykle (zwykle trwające 2-4 tygodnie), w trakcie których zespół realizuje zadania.
2. Kanban: Wizualny system zarządzania pracą, w którym zespół pracuje w sposób ciągły, bez sztywnego podziału na etapy.
3. XP (Extreme Programming): Skupia się na praktykach inżynierskich i bliskiej współpracy z klientem.
4. Lean Software Development: Wywodząca się z metod zarządzania produkcją, stawia na eliminację marnotrawstwa i maksymalizację wartości dostarczanej klientowi.

Metodologia projektowania oprogramowania

- **Kluczowe cechy modelu Agile:**
- Elastyczność – Agile umożliwia szybkie dostosowanie się do zmian w wymaganiach i priorytetach.
- Zaangażowanie klienta – Klient bierze aktywny udział w procesie tworzenia oprogramowania.
- Krótkie cykle dostarczania – Oprogramowanie jest dostarczane w małych, regularnych porcjach.

Agile zyskało popularność w szybko zmieniających się środowiskach, gdzie szybka reakcja na potrzeby klienta i dynamiczne zmiany są kluczowe dla sukcesu projektu.

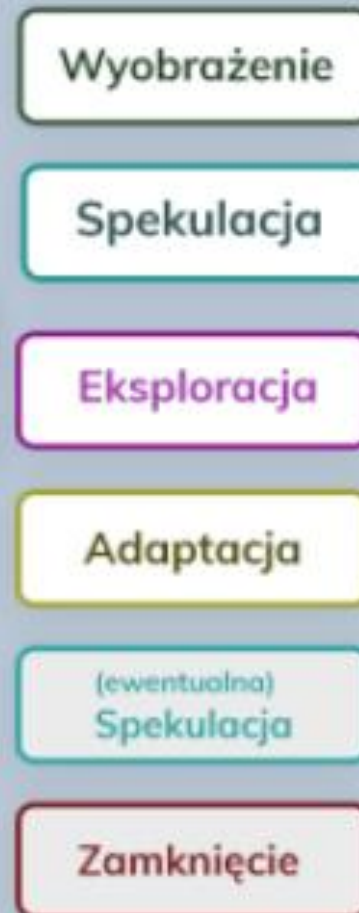
Metodyka Agile

(zwinna)

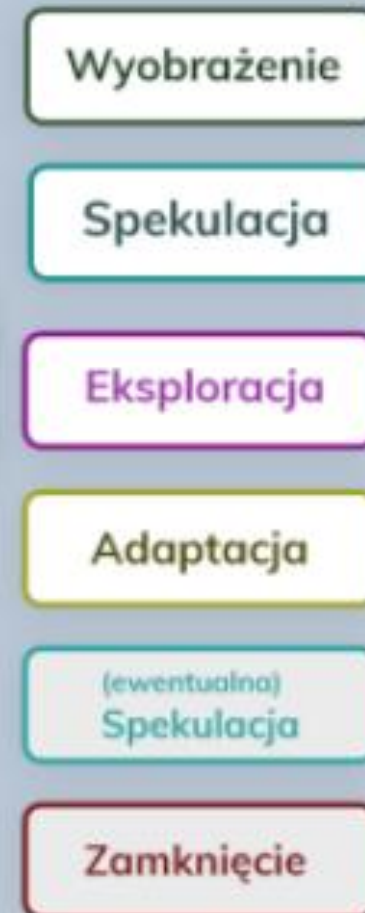
Sprint 1



Sprint 2



Sprint 3



Scrum jako framework w Agile

Scrum jest lekkim, prostym do zrozumienia frameworkiem, ale trudnym do opanowania w pełni. Został zaprojektowany, aby pomóc zespołom w zarządzaniu i organizacji pracy nad złożonymi problemami. Zgodnie z zasadami Agile, Scrum kładzie nacisk na współpracę, ciągłe doskonalenie oraz adaptacyjność do zmian.

Cechy

- Praca przebiega w krótkich, ustalonych iteracjach zwanych sprintami (zwykle 2-4 tygodnie).
- Zespoły koncentrują się na dostarczaniu gotowych, działających elementów produktu na koniec każdego sprintu.
- Proces Scrumowy opiera się na transparentności, inspekcji i adaptacji – trzy filary Scruma, które wspierają jego działanie.

Role w Scrum

1. Product Owner (Właściciel Produktu):

1. Odpowiada za maksymalizację wartości produktu dostarczanego przez zespół.
2. Tworzy i zarządza Product Backlogiem – priorytetyzuje zadania, aby zespół mógł skupić się na najważniejszych elementach.
3. Współpracuje z interesariuszami, aby zrozumieć potrzeby rynku i dostosować produkt do tych wymagań.

2. Scrum Master:

1. Facylitator procesu Scrumowego, dbający o to, aby zespół przestrzegał zasad Scruma.
2. Chroni zespół przed zakłóceniami zewnętrznymi i pomaga usuwać przeszkody (tzw. impedimenty).
3. Edukuje zespół i organizację na temat Scruma, promuje ciągłe doskonalenie.

3. Zespół deweloperski:

1. Samoorganizujący się zespół specjalistów, którzy wspólnie pracują nad realizacją celów sprintu.
2. Składa się zazwyczaj z programistów, testerów, architektów, ale wszyscy członkowie są odpowiedzialni za dostarczenie kompletnego produktu.

Artefakty w Scrumie

Artefakty Scruma zapewniają przejrzystość procesów i pomagają zespołowi skupić się na dostarczaniu wartościowych funkcji.

1. Product Backlog:

1. Zbiór wszystkich zadań, funkcji, usprawnień i poprawek, które mogą być wprowadzone do produktu.
2. Jest dynamiczny – zmienia się w miarę rozwoju produktu i poznawania nowych wymagań.

2. Sprint Backlog:

1. Lista zadań, które zespół wybrał do realizacji w danym sprincie.
2. Jest wycinkiem Product Backlogu i zawiera zadania o najwyższym priorytecie, które zespół zobowiązuje się ukończyć w czasie trwania sprintu.

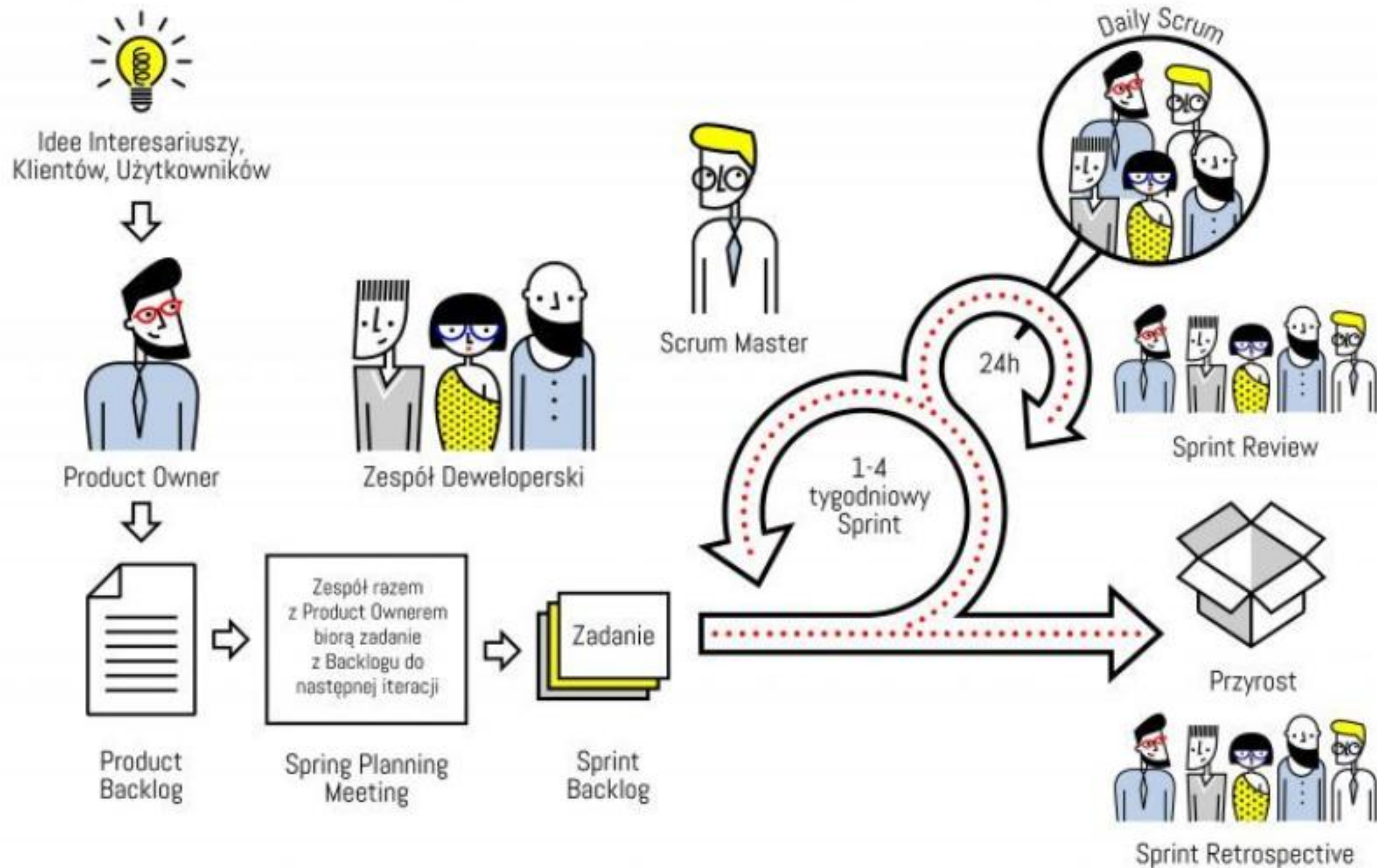
3. Inkrement:

1. Gotowy do wdrożenia produkt lub jego funkcjonalność, która została ukończona podczas sprintu i jest zgodna z ustalonymi kryteriami akceptacji.
2. Każdy inkrement jest przyrostem do istniejącego produktu i musi być działający oraz przetestowany.

Wydarzenia Scrumowe

Scrum zawiera zestaw regularnych spotkań (zwanymi ceremoniami lub wydarzeniami), które pomagają zespołowi w organizacji i synchronizacji pracy.

1. **Sprint:**
 1. Iteracja w Scrumie, która trwa od 1 do 4 tygodni. Na początku sprintu zespół planuje zadania, a na końcu dokonuje przeglądu i retrospektywy.
 2. Każdy sprint kończy się wytworzeniem gotowego do wdrożenia produktu (inkrementu)
2. **Daily Scrum:**
 1. Krótkie, 15-minutowe spotkanie każdego dnia pracy, podczas którego członkowie zespołu synchronizują swoją pracę, omawiają postępy i identyfikują ewentualne przeszkody.
 2. Celem jest bieżące monitorowanie postępu sprintu i dostosowywanie planów
3. **Sprint Review:**
 1. Spotkanie odbywające się pod koniec sprintu, podczas którego zespół prezentuje interesariuszom, co zostało osiągnięte.
 2. Jest okazją do uzyskania informacji zwrotnych i oceny, czy praca zespołu spełnia oczekiwania.
4. **Sprint Retrospective:**
 1. Odbywa się na koniec sprintu, po Sprint Review. Zespół omawia, co poszło dobrze, co mogłoby zostać poprawione oraz jak można usprawnić pracę w przyszłości.
 2. Celem jest ciągłe doskonalenie procesów i organizacji pracy zespołu



Praktyczna symulacja Scruma

Cel: Zastosowanie poznanej wiedzy w praktyce.

Praktyka: Podział studentów na zespoły gdzie, mały projekt IT muszą zrealizować przy użyciu Scruma. Na przykład: "Zaprojektujcie prototyp aplikacji do zarządzania zadaniami".

- Zespół wybiera Product Ownera, Scrum Mastera i deweloperów.
- Tworzą sprint backlog z zadań (np. zaprojektowanie interfejsu, baza danych, etc.).
- Przeprowadzają sprint planning, a następnie symulują daily stand-up.

Podsumowanie: Krótkie omówienie doświadczeń – co poszło dobrze, co było trudne, jakie role odegrały kluczową rolę w synchronizacji zespołu?

Czego powinni nauczyć się studenci po zajęciach:

- Zrozumienie struktury i dynamiki Scruma.
- Umiejętność rozróżniania ról Scrumowych i ich odpowiedzialności.
- Znajomość artefaktów Scrumowych oraz cyklu pracy w sprincie.
- Gotowość do przeprowadzenia sprintu, organizowania spotkań Scrumowych i korzystania z narzędzi do zarządzania Scrumem.